

DB (1~6,14/ 7~13)

1. USERS

사용자의 가장 기본적인 정보입니다.

users

컬럼	타입	제약	설명
id	BIGINT	PK	사용자 ID
nickname	VARCHAR(30)	NOT NULL	화면에 표시할 이름
email	VARCHAR(255)	UNIQUE, NOT NULL	로그인 이메일
password_hash	VARCHAR(255)	NOT NULL	비밀번호
deleted_at	TIMESTAMP	NULL	회원 탈퇴

예:

```
users

id = 1
nickname = "눈눈"
email = "nunnun@example.com"
password_hash = "$2a$..."
```

비밀번호는 절대로 원문을 저장하면 안 되고 BCrypt 등으로 암호화해서 저장합니다.

2.users_devices

컬럼	타입	제약	설명
id	BIGINT	PK	디바이스 ID
user_id	BIGINT	FK, NOT NULL	사용자 ID
fcm_token	VARCHAR(512)	UNIQUE, NOT NULL	FCM 디바이스 토큰
platform	VARCHAR(20)	NOT NULL	ANDROID
created_at	TIMESTAMP	NOT NULL	등록 시각
updated_at	TIMESTAMP	NOT NULL	마지막 수정 시각

3. REFRESH_TOKENS

JWT Refresh Token을 관리합니다.

refresh_tokens

컬럼	타입	제약	설명
id	BIGINT	PK	Token ID
user_id	BIGINT	FK	사용자
token_hash	VARCHAR(255)	NOT NULL	Refresh Token Hash
expires_at	TIMESTAMP	NOT NULL	만료일
revoked_at	TIMESTAMP	NULL	로그아웃/폐기
created_at	TIMESTAMP	NOT NULL	생성일

User 1 : N RefreshToken

여러 기기 로그인을 지원할 가능성까지 생각하면 1:N이 좋습니다.

refresh_tokens 테이블은 **JWT 로그인에서 Access Token이 만료됐을 때 새 Access Token을 발급하기 위해 필요합니다.**

쉽게 말하면 로그인 유지용입니다.

Access Token은 보통 수명이 짧습니다.

Access Token
→ 30분 ~ 몇 시간 정도

Refresh Token
→ 며칠 ~ 몇 주 정도

예를 들어 사용자가 로그인하면:

Access Token
Refresh Token

두 개를 발급합니다.

그 후 Access Token이 만료되면 다시 이메일/비밀번호로 로그인시키는 대신,

Refresh Token
→ 서버 검증
→ 새 Access Token 발급

하는 방식입니다.

그래서 이런 API가 있었죠.

POST /api/v1/auth/reissue

흐름은:

로그인
↓
Access Token + Refresh Token 발급
↓
Access Token 만료
↓
Refresh Token으로 재발급 요청
↓
서버가 Refresh Token 확인
↓
새 Access Token 발급

그런데 **왜 굳이 DB에 저장하느냐**가 핵심입니다.

DB에 저장하면 서버가 Refresh Token을 강제로 무효화할 수 있습니다.

예를 들어 로그아웃하면:

refresh_tokens에서 해당 토큰 삭제 또는 revoked 처리

할 수 있습니다.

그러면 누군가 예전에 받은 Refresh Token을 가지고 있어도 재로그인이 불가능해집니다.

반대로 Refresh Token을 DB에 전혀 저장하지 않고 JWT 자체의 서명만 검사한다면, 토큰의 유효기간이 끝나기 전까지 서버에서 강제로 끊기가 까다로워집니다.

NUNNUN 같은 앱에서는 특히 사용자가 앱을 자주 열 가능성이 있어서 매번 로그인시키기 보다는:

한 번 로그인

→ 로그인 상태 장기간 유지

가 자연스럽습니다. 그래서 Refresh Token 구조를 쓰는 편이 좋습니다.

다만 **반드시 별도 테이블이 필요한 것은 아닙니다.**

프로젝트가 해커톤/MVP 규모이고 한 계정당 한 기기 정도만 생각한다면 더 단순하게:

```
users
- id
- email
- password
- refresh_token
```

처럼 `users` 에 Refresh Token 하나를 넣을 수도 있습니다.

하지만 지금 NUNNUN은 이미 `users_devices` 를 두고 있고 여러 기기 가능성을 열어두고 있기 때문에 저는 별도 `refresh_tokens` 테이블을 제안한 겁니다.

정리하면:

```
Access Token
= API를 사용할 수 있는 짧은 출입증

Refresh Token
= 출입증이 만료됐을 때 새 출입증을 받을 수 있는 갱신권

refresh_tokens 테이블
= 그 갱신권이 아직 유효한지 서버가 관리하는 곳
```

그리고 **학교 프로젝트/해커톤 수준에서 복잡도를 낮추고 싶다면 `refresh_tokens` 테이블 없이 시작해도 됩니다.** 처음에는 Access Token + Refresh Token 구조만 만들고, Refresh Token 하나를 `users` 또는 Redis에 관리하는 방식으로 단순화할 수도 있습니다.

4. FIXED_SCHEDULES

My 그룹의 고정 시간표입니다.

`fixed_schedules`

컬럼	타입	제약	설명
id	BIGINT	PK	일정 ID
user_id	BIGINT	FK	사용자

컬럼	타입	제약	설명
title	VARCHAR(100)	NOT NULL	일정 이름
day_of_week	VARCHAR(10)	NOT NULL	MONDAY 등
start_time	TIME	NOT NULL	시작
end_time	TIME	NOT NULL	종료
created_at	TIMESTAMP	NOT NULL	생성
updated_at	TIMESTAMP	NOT NULL	수정

예:

```
id: 1
user_id: 3
title: 인공지능
day_of_week: MONDAY
start_time: 09:00
end_time: 10:30
```

5. DAILY_ROUTINES

여기가 My 그룹에서 가장 중요한 테이블 중 하나입니다.

하루의 예정된 생활 정보를 저장합니다.

daily_routines

컬럼	타입	제약	설명
id	BIGINT	PK	ID
user_id	BIGINT	FK	사용자
routine_date	DATE	NOT NULL	날짜
target_bed_time	TIME	NULL	목표 취침시간
target_wake_time	TIME	NULL	목표 기상시간
estimated_return_time	TIME	NULL	예상 귀가시간
updated_at	TIMESTAMP	NOT NULL	수정일
estimated_return_at	TIMESTAMP	NULL	시간 기준점

중요한 UNIQUE:

```
UNIQUE(user_id, routine_date)
```

즉 한 사용자는 하루에 하나의 DailyRoutine만 가집니다.

예:

```
2026-08-11
```

```
target_bed_time = 23:30  
target_wake_time = 08:00  
estimated_return_time = 20:00
```

6. SLEEP_SESSIONS

사용자가 잠들기 버튼을 누른 기록입니다.

sleep_sessions

컬럼	타입	제약	설명
id	BIGINT	PK	수면 ID
user_id	BIGINT	FK	사용자
sleep_date	DATE	NOT NULL	수면 기준 날짜
started_at	TIMESTAMP	NOT NULL	잠들기 버튼 누른 시간
created_at	TIMESTAMP	NOT NULL	생성일

현재 기상 완료를 관리하지 않기로 했으므로:

```
ended_at  
wake_at
```

같은 컬럼은 일단 필요 없습니다.

룸메이트 화면의:

```
4시간째 취침 중
```

은 DB에 4시간 을 저장하는 게 아니라

현재 시간 - started_at

으로 계산합니다.

7. SLEEP_FEEDBACKS

하루 1번 기록하는 수면 만족도입니다.

sleep_feedbacks

컬럼	타입	제약	설명
id	BIGINT	PK	ID
user_id	BIGINT	FK	사용자
feedback_date	DATE	NOT NULL	날짜
score	VARCHAR(20)	NOT NULL	만족도
created_at	TIMESTAMP	NOT NULL	생성일

Score:

VERY_BAD
BAD
NORMAL
GOOD
VERY_GOOD

중요:

UNIQUE(user_id, feedback_date)

이걸 걸어서 **하루 1회**를 DB 레벨에서도 보장합니다.

8. WAKE_GROUPS

깨우기 그룹입니다.

wake_groups

컬럼	타입	제약	설명
id	BIGINT	PK	그룹 ID
name	VARCHAR(50)	NOT NULL	그룹 이름
invite_code	VARCHAR(6)	UNIQUE	초대코드
creator_id	BIGINT	FK	생성자
created_at	TIMESTAMP	NOT NULL	생성일

예:

제주도 여행
초대코드: A7B2CD

9. WAKE_GROUP_MEMBERS

사용자와 깨우기 그룹은 N:M 관계입니다.

User N : M WakeGroup

따라서 중간 테이블이 필요합니다.

wake_group_members

컬럼	타입	제약	설명
id	BIGINT	PK	ID
wake_group_id	BIGINT	FK	깨우기 그룹
user_id	BIGINT	FK	사용자
slot_no	SMALLINT	NOT NULL	1~12
joined_at	TIMESTAMP	NOT NULL	참여일

제약:

```

UNIQUE(wake_group_id, user_id)

CHECK(slot_no BETWEEN 1 AND 12)

UNIQUE(wake_group_id, slot_no)

```


이 구조가 꽤 중요합니다.

그룹 최대 인원 12명을 **DB 차원에서도 제한**할 수 있습니다.

```
slot 1
slot 2
...
slot 12
```

까지만 존재할 수 있기 때문입니다.

그리고 프론트에서:

```
1~4   → Page 1
5~8   → Page 2
9~12  → Page 3
```

로 사용할 수도 있습니다.

10. WAKE_REQUESTS

누가 누구를 깨웠는지를 관리합니다.

wake_requests

컬럼	타입	제약	설명
id	BIGINT	PK	요청 ID
wake_group_id	BIGINT	FK	그룹
sender_id	BIGINT	FK	깨운 사람
receiver_id	BIGINT	FK	깨워지는 사람
status	VARCHAR(20)	NOT NULL	상태
requested_at	TIMESTAMP	NOT NULL	요청 시간
created_at	TIMESTAMP	NOT NULL	생성일

Status 후보:

```
SENT
VERIFIED
EXPIRED
```

11. WAKE_PROOFS

기상 인증사진입니다.

wake_proofs

컬럼	타입	제약	설명
id	BIGINT	PK	인증 ID
wake_request_id	BIGINT	UNIQUE, FK	깨우기 요청
image_object_key	VARCHAR(512)	NOT NULL	S3 파일 키
verified_at	TIMESTAMP	NOT NULL	인증시간
expires_at	TIMESTAMP	NOT NULL	삭제 예정시간

예:

```
verified_at = 07:32
expires_at = 15:32
```

인증사진 삭제

인증 완료 시:

```
expires_at = verified_at + 8시간
```

expires_at이 지난 인증사진은 삭제한다.

삭제 대상:

- S3 인증사진
- wake_proofs 데이터

그리고 깨우기 30분 쿨타임은 별도 컬럼이 없어도 됩니다.

```
verified_at + 30분
```

으로 계산할 수 있습니다.

즉:

07:32 인증

07:32 ~ 08:02

깨우기 불가능

08:02 이후
깨우기 가능

15:32
사진 실제 삭제

입니다.

12. ROOMMATE_GROUPS

룸메이트 그룹입니다.

roommate_groups

컬럼	타입	제약	설명
id	BIGINT	PK	그룹 ID
name	VARCHAR(50)	NOT NULL	그룹 이름
invite_code	VARCHAR(6)	UNIQUE	초대코드
creator_id	BIGINT	FK	생성자
status	VARCHAR(20)	NOT NULL	WAITING / ACTIVE
created_at	TIMESTAMP	NOT NULL	생성일

상태:

WAITING
→ 한 명만 있음

ACTIVE
→ 룸메이트 참여 완료

13. ROOMMATE_GROUP_MEMBERS

여기서는 조금 특별하게 설계하는 게 좋습니다.

roommate_group_members

컬럼	타입	제약	설명
id	BIGINT	PK	ID
roommate_group_id	BIGINT	FK	그룹

컬럼	타입	제약	설명
user_id	BIGINT	FK, UNIQUE	사용자
slot_no	SMALLINT	NOT NULL	2

제약:

```

UNIQUE(user_id)

CHECK(slot_no IN (1, 2))

UNIQUE(roommate_group_id, slot_no)

UNIQUE(roommate_group_id, user_id)

```

이 설계의 장점이 큼니다.

사용자당 룸메이트 그룹 하나

```

UNIQUE(user_id)

```

덕분에 한 사용자가:

```

룸메이트 그룹 A
+
룸메이트 그룹 B

```

에 동시에 들어갈 수 없습니다.

최대 2명

```

slot_no = 1
slot_no = 2

```

만 허용하기 때문에 세 번째 사용자는 DB에서도 들어갈 수 없습니다.

14. ROOMMATE_COMPLAINTS

불만사항입니다.

roommate_complaints

컬럼	타입	제약	설명
id	BIGINT	PK	ID
roommate_group_id	BIGINT	FK	그룹
author_id	BIGINT	FK	작성자
target_user_id	BIGINT	FK	대상
content	TEXT	NOT NULL	원문
created_at	TIMESTAMP	NOT NULL	생성일

예:

```
id = 1
author
= 눈눈

target
= 은지

content
= "밤에 들어올 때 문을 너무 세게 달아서 자꾸 깨요."
```

불만사항 수정 API

PATCH /roommate-complaints/{complaintId}

complaintId는 roommate_complaints.id를 의미한다.
해당 id의 불만사항 content를 수정한다.

예:

PATCH /roommate-complaints/1

```
Request
{
  "content": "밤에 문 달는 소리가 커서 잠에서 깨요."
}
```

하지만 이 원문은 은지에게 API Response로 보내면 안 됩니다.

흐름은:

```
Complaint DB
↓
OpenAI
↓
순화
```

↓
행동 매뉴얼

불만사항 원문은 roommate_complaints에 저장한다.

불만 등록 또는 수정 시 OpenAI가 해당 대상 사용자의 행동 매뉴얼을 생성하거나 갱신한다.

생성된 행동 매뉴얼은 roommate_behavior_manuals에 저장한다.
UNIQUE(roommate_group_id, target_user_id)를 통해
그룹 내 대상 사용자별 현재 매뉴얼을 하나만 유지한다.

불만 수정 API의 요청은 `content` 만 받도록 합니다.

```
{  
  "content": "밤에 문 닫는 소리가 커서 잠에서 깨요."  
}
```

입니다.

가능하면 이 데이터는 민감한 내용이므로 추후 **암호화 저장**도 고려하는 게 좋습니다.

15. ROOMMATE_BEHAVIOR_MANUALS

컬럼	타입	제약	설명
id	BIGINT	PK	행동 매뉴얼 ID
roommate_group_id	BIGINT	FK, NOT NULL	룸메이트 그룹 ID
target_user_id	BIGINT	FK, NOT NULL	행동 매뉴얼 대상 사용자 ID
content	TEXT	NOT NULL	OpenAI가 생성한 행동 매뉴얼 내용
generated_at	TIMESTAMP	NOT NULL	매뉴얼 최초 생성 시각
updated_at	TIMESTAMP	NOT NULL	매뉴얼 마지막 수정 시각

그리고 가장 중요한 제약:

```
UNIQUE(roommate_group_id, target_user_id)
```

를 겁니다.

즉 한 룸메이트 그룹에서 사용자 한 명당 현재 행동 매뉴얼 하나만 존재하게 하는 겁니다.

16. NOTIFICATIONS

NUNNUN은 알림이 핵심 기능이어서 알림 기록은 남기는 게 좋습니다.

notifications

컬럼	타입	제약	설명
id	BIGINT	PK	알림 ID
user_id	BIGINT	FK	받는 사용자
type	VARCHAR(50)	NOT NULL	알림 종류
title	VARCHAR(100)	NOT NULL	제목
body	VARCHAR(500)	NULL	내용
reference_id	BIGINT	NULL	관련 객체 ID
scheduled_at	TIMESTAMP	NULL	예약시간
sent_at	TIMESTAMP	NULL	발송시간
status	VARCHAR(20)	NOT NULL	상태
created_at	TIMESTAMP	NOT NULL	생성일

Type:

```
daily_routines.id  
wake_requests.id  
sleep_sessions.id  
daily_routines.id
```

Status:

```
PENDING  
SENT  
FAILED  
CANCELLED
```

예를 들어 취침 알림:

```
23:30 목표 취침
```

22:30
BEDTIME_REMINDER 발송

잠들기 버튼을 안 누르면:

00:00
다음 알림

01:30
다음 알림

처럼 관리할 수 있습니다.

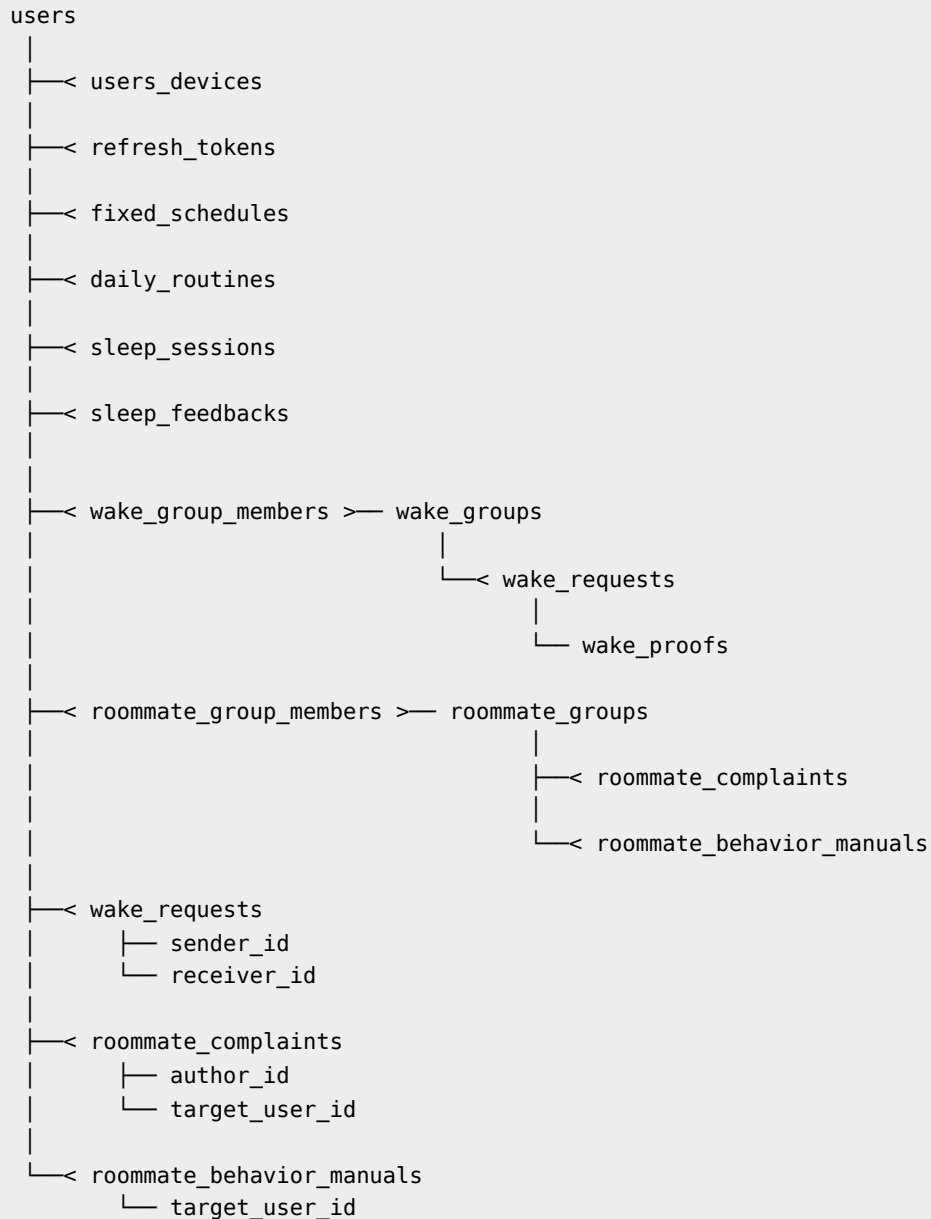
17. 최종 테이블 목록

Domain	Table
User	users
Device	users_devices
Auth	refresh_tokens
Schedule	fixed_schedules
Routine	daily_routines
Sleep	sleep_sessions
Sleep	sleep_feedbacks
Wake	wake_groups
Wake	wake_group_members
Wake	wake_requests
Wake	wake_proofs
Roommate	roommate_groups
Roommate	roommate_group_members
Roommate	roommate_complaints
Roommate	roommate_behavior_manuals
Notification	notifications

총 16개 테이블입니다.

18. 핵심 FK 관계

최종적으로 핵심 관계만 보면:



`users_devices` 는 `user_id` 로 `users` 를 참조하고, FCM 토큰을 저장하는 구조입니다.

조금 더 정확하게 FK를 하나씩 풀어쓰면 다음과 같습니다.

```
users.id
├─ users_devices.user_id
├─ refresh_tokens.user_id
├─ fixed_schedules.user_id
├─ daily_routines.user_id
```

```

├─ sleep_sessions.user_id
├─ sleep_feedbacks.user_id
├─ wake_groups.creator_id
├─ wake_group_members.user_id
├─ wake_requests.sender_id
├─ wake_requests.receiver_id
├─ roommate_groups.creator_id
├─ roommate_group_members.user_id
├─ roommate_complaints.author_id
├─ roommate_complaints.target_user_id
├─ roommate_behavior_manuals.target_user_id

```

```

wake_groups.id
├─ wake_group_members.wake_group_id
├─ wake_requests.wake_group_id

```

```

wake_requests.id
├─ wake_proofs.wake_request_id

```

```

roommate_groups.id
├─ roommate_group_members.roommate_group_id
├─ roommate_complaints.roommate_group_id
├─ roommate_behavior_manuals.roommate_group_id

```

`roommate_behavior_manuals` 는 룸메이트 그룹과 대상 사용자를 동시에 참조하고, 그룹 내 사용자별 현재 매뉴얼을 하나만 유지하기 위해 아래 제약을 둡니다.

```

UNIQUE(roommate_group_id, target_user_id)

```

그리고 `roommate_complaints` 는 작성자와 대상 사용자를 모두 `users` 로 연결합니다.

```

roommate_complaints.author_id
→ users.id

roommate_complaints.target_user_id
→ users.id

```

Wake도 마찬가지입니다.

```

wake_requests.sender_id
→ users.id

```

```
wake_requests.receiver_id  
→ users.id
```

즉 전체 구조를 한 줄로 요약하면:

```
User  
├─ 개인 데이터  
│   ├── Device  
│   ├── Token  
│   ├── Schedule  
│   ├── Routine  
│   └─ Sleep  
├─ Wake Group  
│   ├── Member  
│   ├── WakeRequest  
│   └─ WakeProof  
└─ Roommate Group  
    ├── Member  
    ├── Complaint  
    └─ BehaviorManual
```